MISIÓN SMARTPARKING

PARTE 1 | Diseño arquitectónico de una solución simple

MODALIDAD	DURACIÓN	EQUIPO	ENTREGA
Grupal	120 minutos	5 integrantes	Un PDF por equipo

La empresa SmartParking administra reservas de vehículos y necesita una solución segura y extensible. El objetivo no es programar el sistema, sino diseñar una arquitectura que explique componentes, interfaces y decisiones técnicas defendibles.

01 El sistema que hay que diseñar

La aplicación debe administrar solamente tres entidades persistentes en una base de datos relacional:



Figura 1 · Modelo conceptual orientativo. No es un diagrama UML de componentes.

Entidad	Datos mínimos
Cliente	id, nombre, email, teléfono, identityUserId
Vehículo	id, matrícula, marca, modelo, clienteId
Reserva	id, clienteId, vehiculoId, inicio, fin, estado

Reglas: un vehículo pertenece a un cliente; una reserva usa un vehículo de ese cliente; no se permiten reservas activas solapadas del mismo vehículo.

DTOs mínimos de intercambio

Los siguientes contratos son orientativos y fijan los datos mínimos. Pueden agregar campos o DTOs de entrada/salida y deberán explicar cómo se utilizan en las interfaces. Los identificadores son ilustrativos.

DTO	Campos mínimos y tipos
ClienteDTO	id: UUID; nombre: string; email: string; telefono: string; identityUserId: string
VehiculoDTO	id: UUID; matricula: string; marca: string; modelo: string; clienteId: UUID
ReservaDTO	id: UUID; clienteId: UUID; vehiculoId: UUID; inicio: datetime; fin: datetime; estado: string (ACTIVA, CANCELADA)
LoginRequest / AuthResponse	email: string; password: string / accessToken: JWT;

	expiresIn: number
EventoAuditoriaDTO	eventId: UUID; tipo: string; entidad: string; entidadId: UUID; userId: string; fechaHora: datetime

El componente de autenticación debe diseñarse por el equipo. Flujo esperado: credenciales → autenticación contra storage de usuarios → emisión de JWT → solicitud de negocio con Bearer token → validación del token y autorización (rol o pertenencia). Se permite proponer componentes internos adicionales, pero no dar por resuelta la identidad mediante un proveedor externo preexistente.

02 Funcionalidades obligatorias

ID	Capacidad	Acceso
RF1	CRUD de clientes	Administrador
RF2	CRUD de vehículos	Cliente (propios) / Administrador
RF3	Crear, modificar, cancelar y consultar reservas	Cliente (propias) / Administrador
RF4	Consulta de reservas propias	Cliente autenticado
RF5	Consulta global de reservas con filtros por cliente, estado y fechas	Administrador

En RF4, el cliente se determina a partir de la identidad autenticada: no alcanza con recibir un clienteId desde la solicitud. RF5 requiere verificar el rol ADMIN.

03 Restricciones de arquitectura y seguridad

- El equipo deberá DISEÑAR el componente de autenticación y autorización como parte de su solución, no asumir que existe un servicio externo ya implementado. Debe definir sus interfaces, responsabilidades, dependencias y mecanismo JWT. No es necesario programarlo. El componente debe autenticar credenciales contra su propio repositorio de usuarios, emitir JWT y permitir validar tokens y permisos de acceso.
- El almacenamiento de usuarios, credenciales y roles DEBE ser independiente del almacenamiento relacional de negocio (Clientes, Vehículos y Reservas), porque pertenecen a componentes con responsabilidades diferentes. No se admite una misma base de datos para ambas responsabilidades; pueden elegir la tecnología de almacenamiento de identidad.
- La identidad del JWT debe vincularse con Cliente.identityUserId. Para consultar reservas propias se obtiene el cliente desde el usuario autenticado y no se confía en un clienteId enviado por el cliente. La consulta global exige el rol ADMIN. El equipo deberá explicar en su diseño dónde se verifican la firma, vigencia y permisos del JWT. No deben almacenarse contraseñas en Cliente.
- Toda alta, modificación o cancelación exitosa debe generar una notificación a un componente de registro de eventos, responsable de conservar un log independiente. El mecanismo de comunicación será una decisión del equipo.
- La base operacional es relacional. Los equipos eligen tecnologías y mecanismos de integración, siempre que puedan justificar su uso.

04 Mapa conceptual del problema

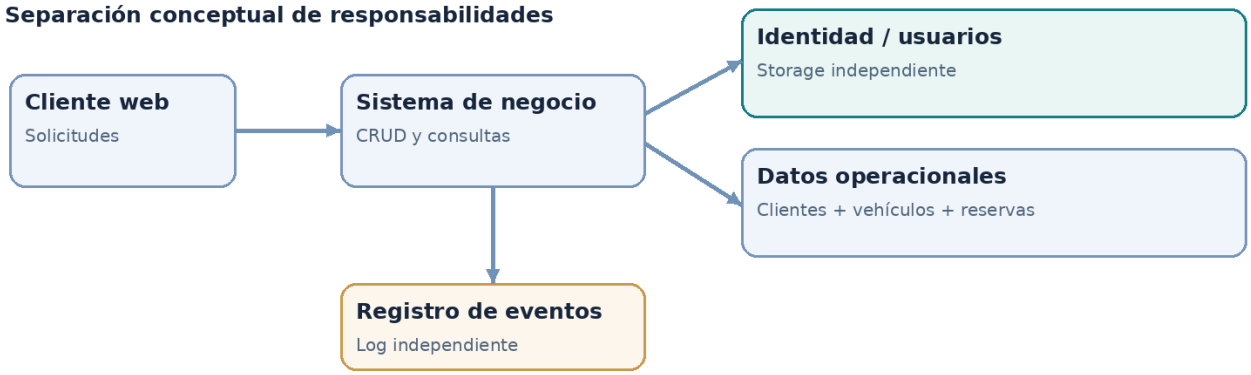


Figura 2 · Mapa de responsabilidades, deliberadamente genérico. No establece componentes UML definitivos, interfaces ni tecnologías.

05 Atributos de calidad

Atributo	Condición a contemplar
Performance	500 solicitudes/s en consultas; p95 < 300 ms en condiciones normales.
Seguridad	JWT válido, rol ADMIN para consulta global y propiedad del recurso para consulta propia.
Modificabilidad	Cambiar el mecanismo de registro de eventos sin modificar la lógica de CRUD.
Integrabilidad	La API debe interactuar con identidad y registro de eventos mediante contratos explícitos.

06 Actividades y entregables

Actividad	Qué se entrega	Puntos
A1 · Componentes	Diagrama UML de componentes: responsabilidades, interfaces provistas/requeridas, dependencias, persistencias separadas y comunicación con el log.	45
A2 · Calidad	Dos escenarios: performance e integrabilidad/modificabilidad. Para cada uno: estímulo, respuesta medible, táctica SAP y ubicación en el diseño.	25
A3 · ADR-01	Decisión sobre cómo comunicar las operaciones exitosas al componente de eventos; al menos dos alternativas, consecuencias y atributos afectados.	25
A4 · Revisión	Coherencia entre diagrama, escenarios, tácticas y ADR.	5

Las herramientas y tecnologías seleccionadas deben identificarse y justificarse brevemente dentro del diagrama o del ADR. No se requiere implementar código, endpoints ni SQL.

07 Rúbrica de corrección

Criterio	Se valorará especialmente
Diagrama UML (45)	Partición adecuada, interfaces y dependencias; componente de autenticación/autorización diseñado por el equipo (login, emisión y validación JWT); control de permisos por rol y pertenencia; repositorio de usuarios independiente de la base de Clientes, Vehículos y Reservas; componente de log separado. Los esquemas del enunciado son solo contexto, no una solución para reproducir.
Calidad (25)	Escenarios verificables, tácticas arquitectónicas correctamente vinculadas con los atributos y su aplicación concreta en el diseño.
ADR (25)	Problema claro, alternativas reales, justificación y consecuencias; coherencia con la solución dibujada.
Consistencia (5)	Requisitos, interfaces, mecanismos y decisiones no se contradicen.

IMPORTANTE SOBRE LOS DIAGRAMAS DEL ENUNCIADO: son esquemas de contexto deliberadamente genéricos y NO constituyen una arquitectura de referencia ni una solución UML. No se evaluará copiarlos ni reproducir su cantidad de cajas. La nota se asignará al diagrama UML de componentes elaborado por el equipo, su nivel de detalle, la claridad de interfaces, la justificación de dependencias y la trazabilidad con los requisitos.

IMPORTANTE EN LA RÚBRICA: los diagramas suministrados son genéricos y no constituyen una solución de componentes. Se evaluará que los equipos diseñen el componente de autenticación propio, con interfaces y JWT, y reflejen la separación física/lógica del storage de usuarios respecto de la base operacional. También deberán mostrar el vínculo entre identidad autenticada y Cliente para aplicar permisos por propiedad y por rol.

Bibliografía de referencia

Bass, L., Clements, P. y Kazman, R. Software Architecture in Practice, 4.ª edición. En particular: escenarios de calidad y tácticas, performance, seguridad, modificabilidad, integrabilidad e interfaces de software.